

LCD Multi Display for MicroBlocks

Educational Examples Guide

Library v1.3

HD44780 16x02 / 20x04 displays with I2C interface

Author: ZLITNI Hsen

This guide presents eight progressive learning topics, implemented through ten MicroBlocks project files, to introduce the main features of the LCD Multi Display library. The library can:

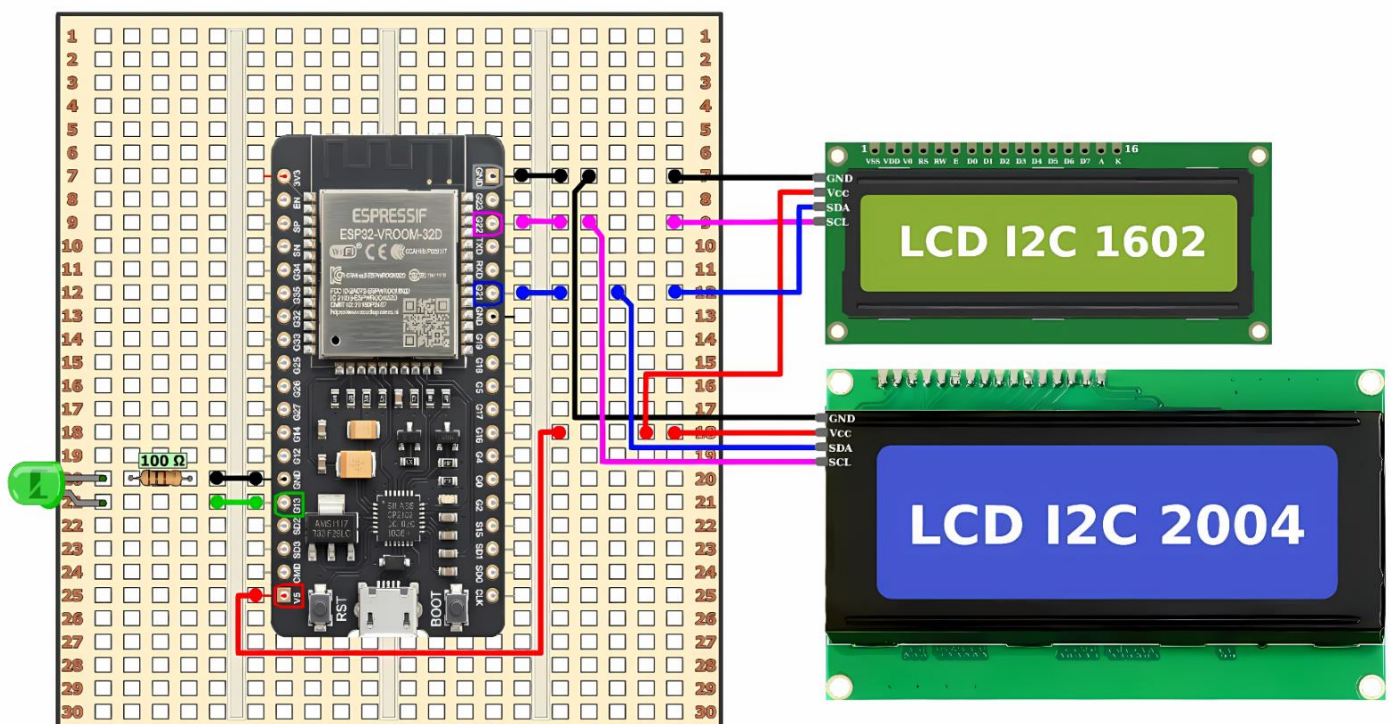
- use 16x02 and 20x04 LCD displays;
- control up to two independent displays;
- use automatic I2C detection or a fixed address;
- print, center, clear and scroll text;
- use symbols and custom characters;
- automatically display several accented characters, ° and Ω;
- provide Clock and Alarm functions;
- run a button-controlled Countdown with Ready, Start, Pause, Resume and restart with the same duration.

Important: run Init before using the LCD display blocks. After a general MicroBlocks Stop, a new Init is required. To use two LCDs simultaneously, they must have different physical I2C addresses.

Wiring:

Example wiring is shown with an ESP32 board. SDA and SCL pins may vary depending on the board. The two LCDs must use different I2C addresses.

The examples in this guide use 0x27 and 0x26; other supported I2C addresses can also be used.

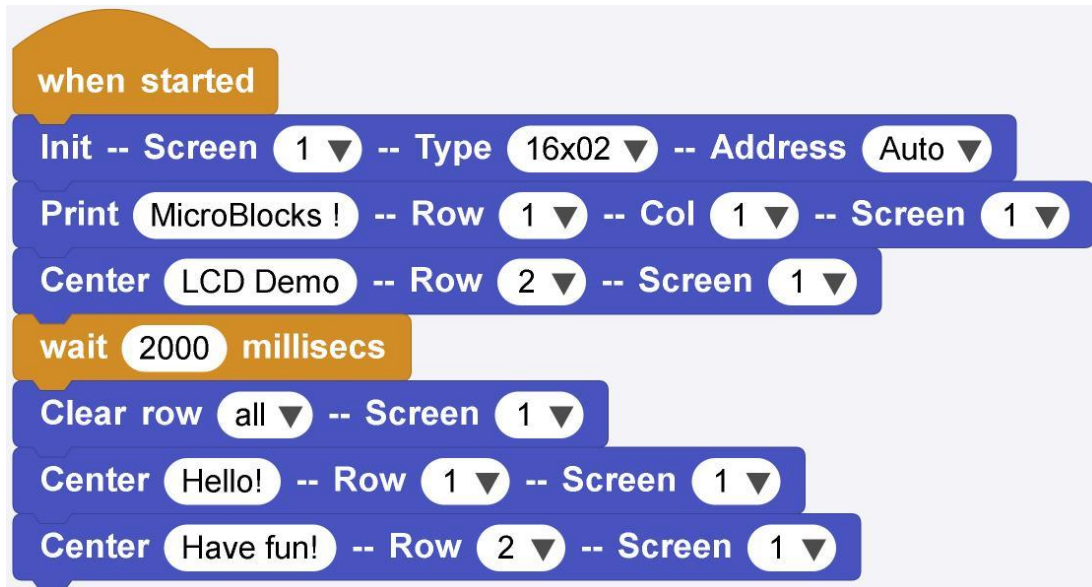


Example 1: Hello LCD (01_Hello_LCD.ubp).

Objective

Learn the minimum steps required to use an LCD: initialization, printing at a position, centering text and clearing the display.

Program



Result

At startup



After two seconds

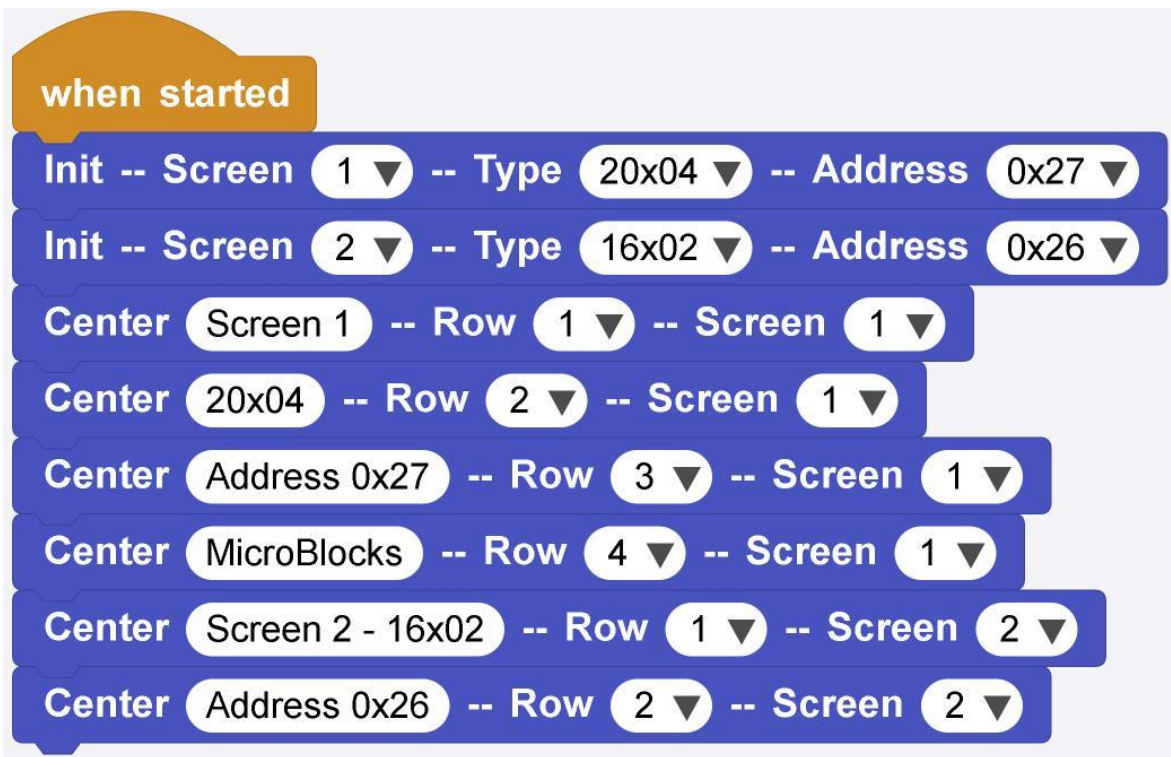


Example 2: Two LCDs (02_Two_LCDs.ubp).

Objective

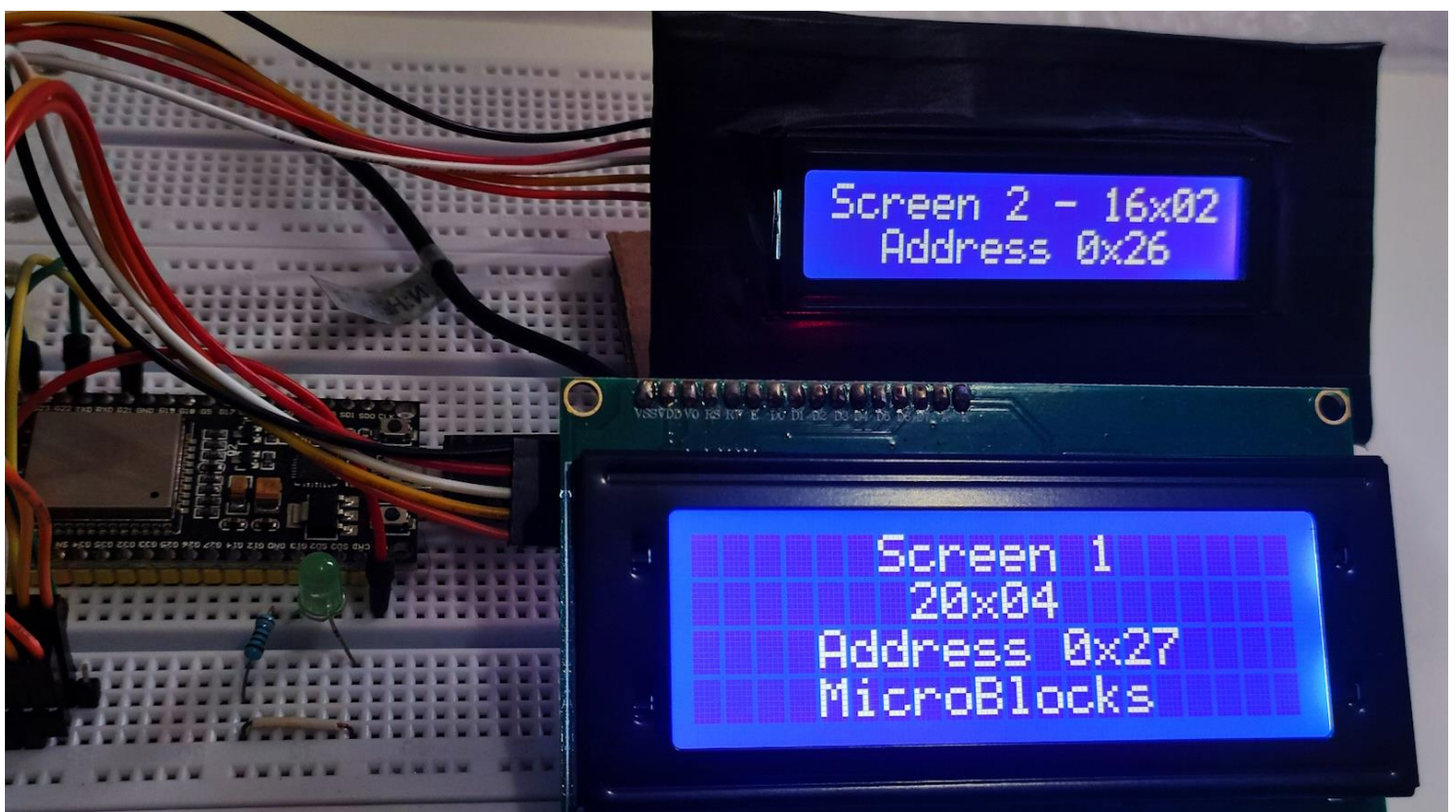
Use a 20x04 and a 16x02 LCD simultaneously. Each display has its own state, dimensions and physical I2C address.

Program



Note: The addresses 0x27 and 0x26 are specific to this example. Your LCD addresses may be different.

Result



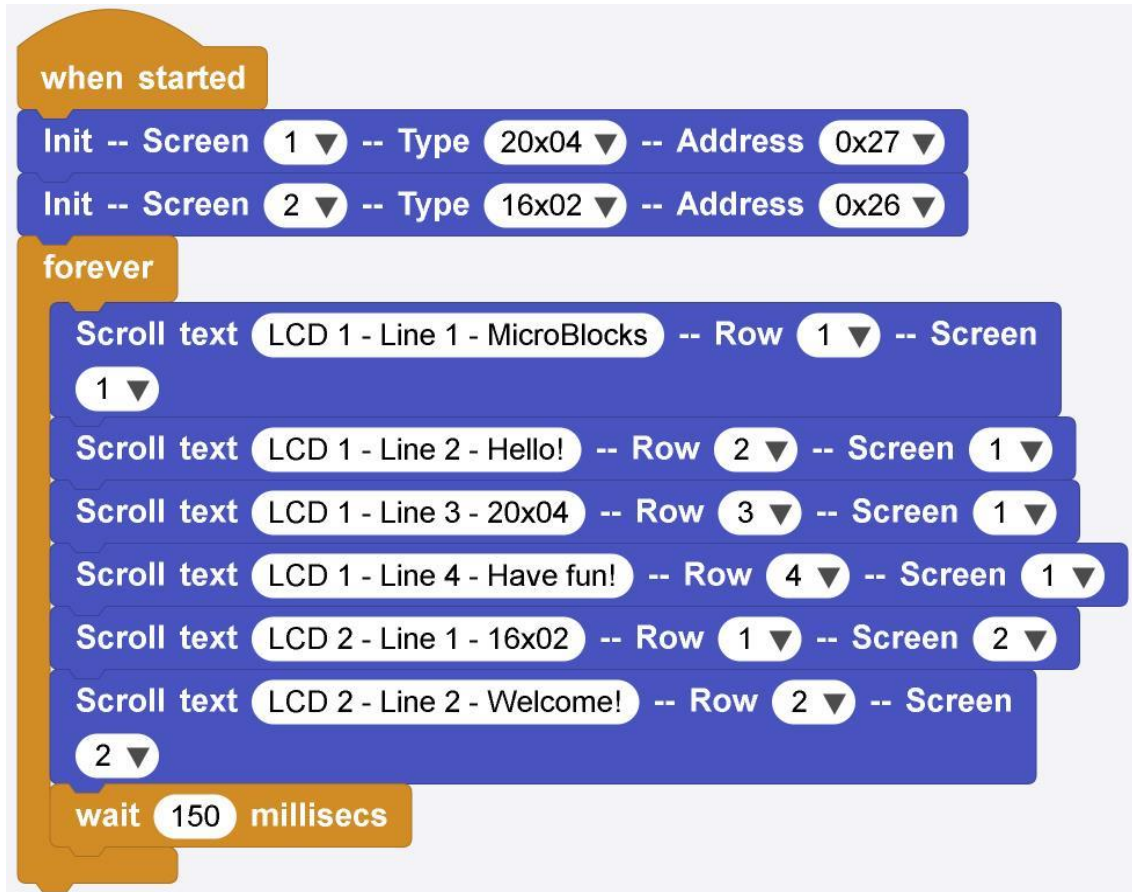
Example 3: Six Independent Scrolls (03_Six_Independent_Scrolls.ubp).

Objective

Demonstrate an important feature of LCD Multi Display: every LCD row has its own independent Scroll state.

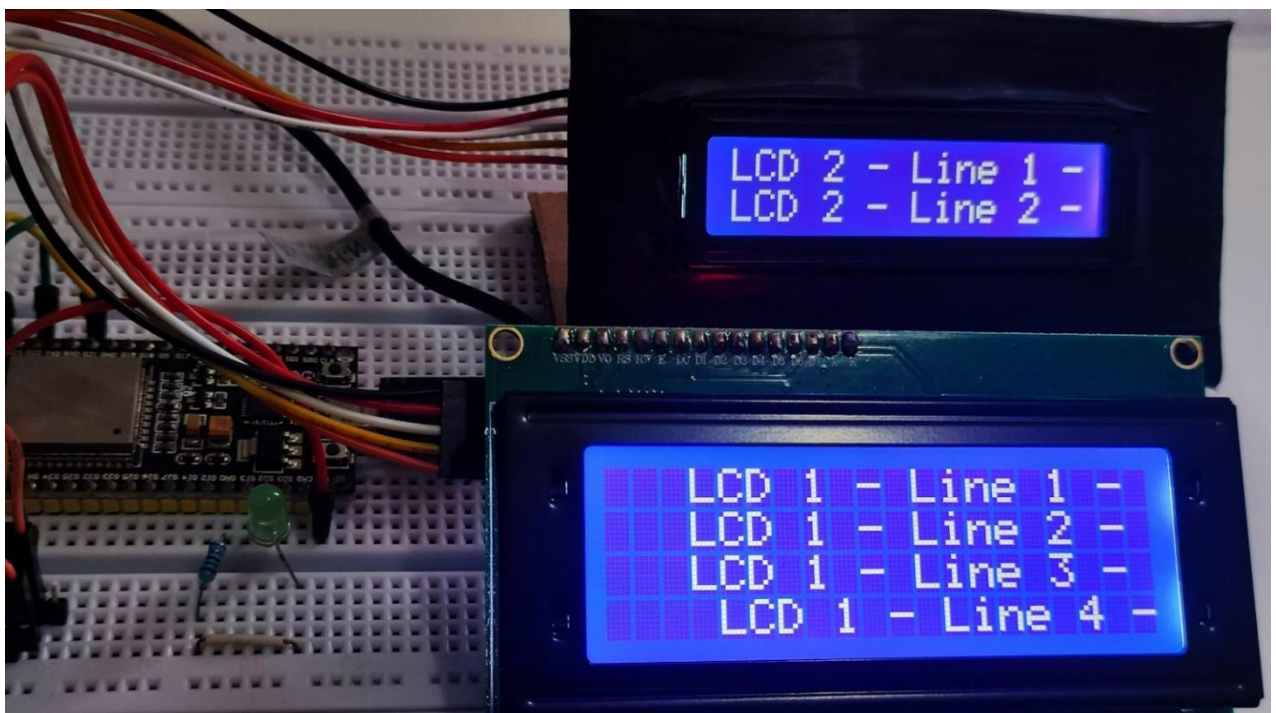
A 20x04 can therefore run four independent scrolling lines while a 16x02 runs two more, for a total of up to six simultaneous scrolling lines.

Program



The wait time controls the scrolling speed. Try changing 150 ms to see the effect.

Result

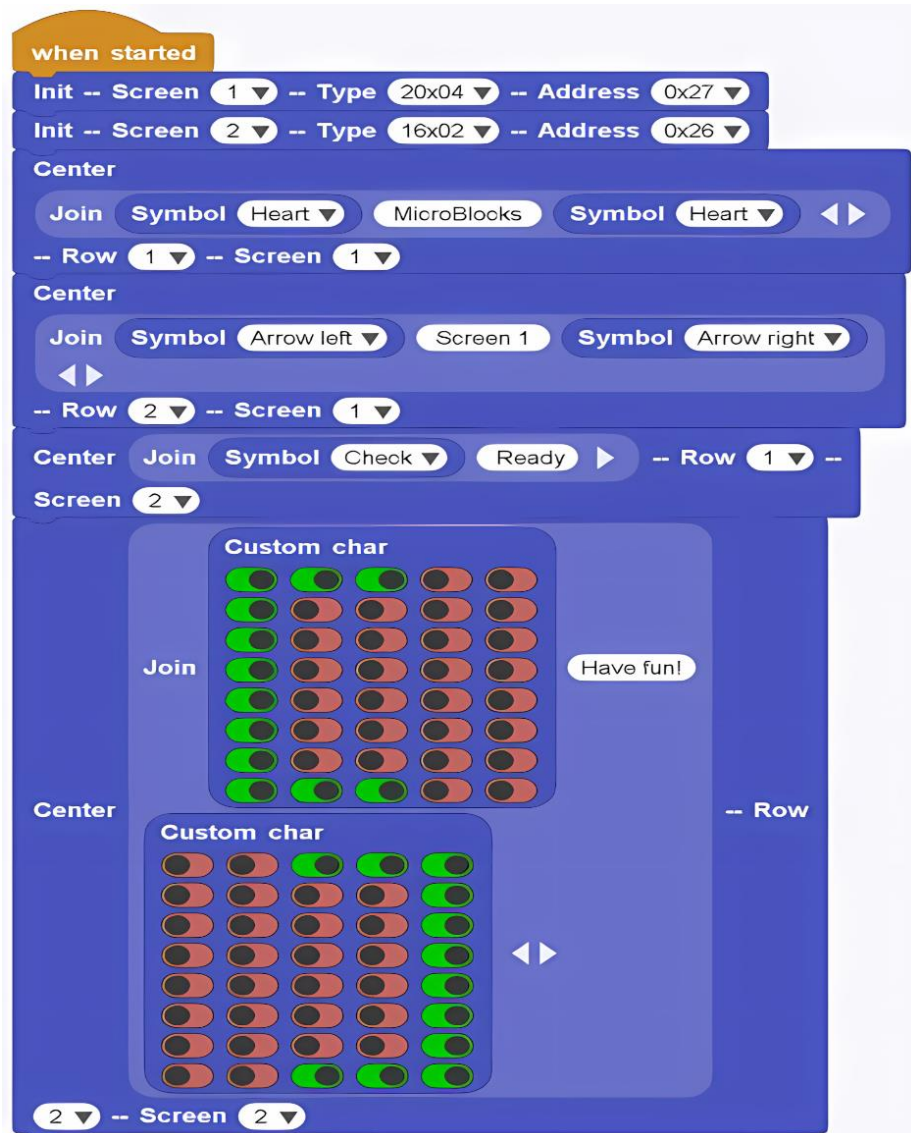


Example 4: Symbols & Custom Chars (04_Symbols_Custom_Chars.ubp).

Objective

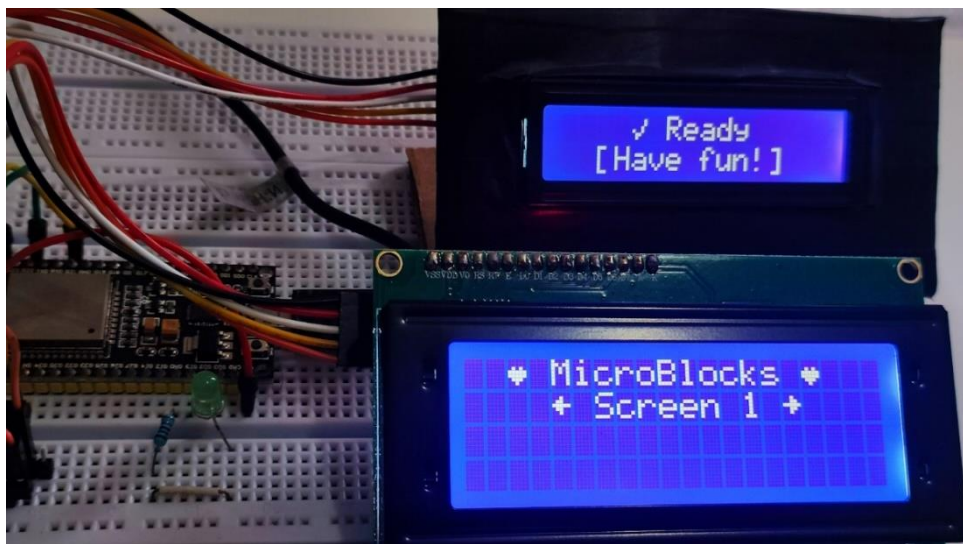
Combine symbols and custom 5x8 characters with normal text, and understand that Symbol and Custom char are not associated with a Screen until they are displayed.

Program



Note: Each LCD has 8 shared custom-character slots (CGRAM). These slots are shared by Symbol, Custom char, Draw char, supported accented characters, ° and Ω. An identical pattern reuses the same slot.

Result

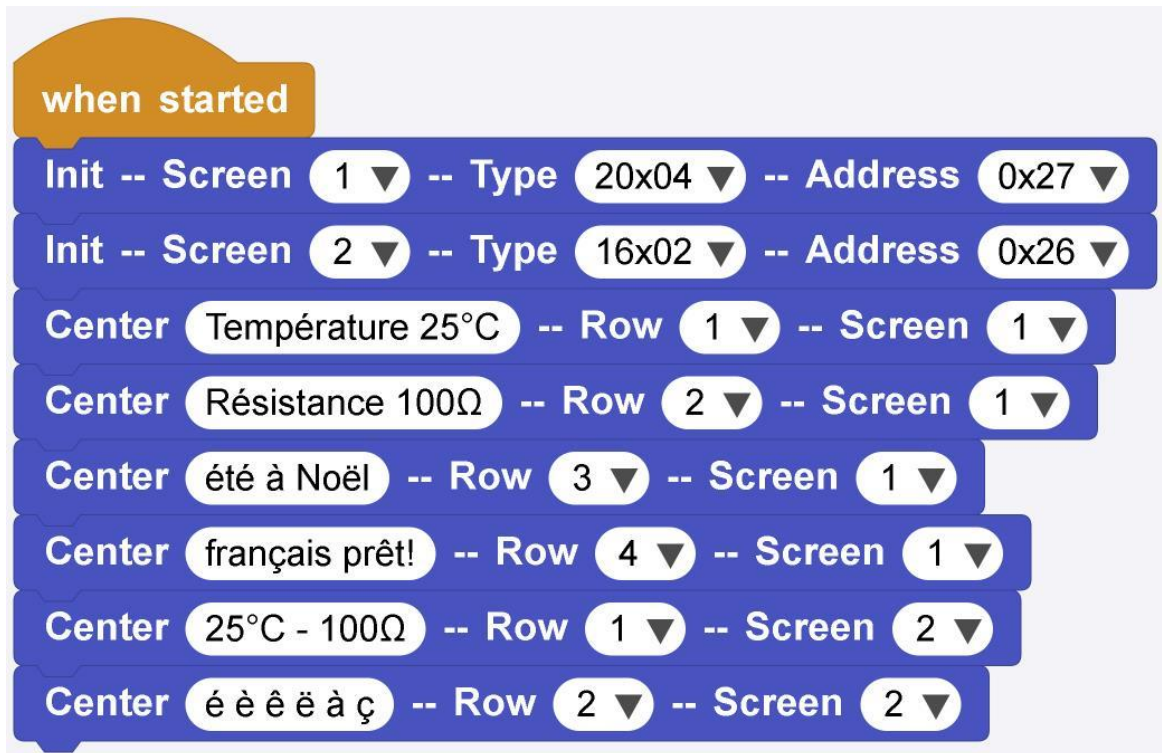


Example 5: Accents & Special Characters (05_Accents_Special_Chars.ubp).

Objective

Display accented and technical characters directly inside normal text.

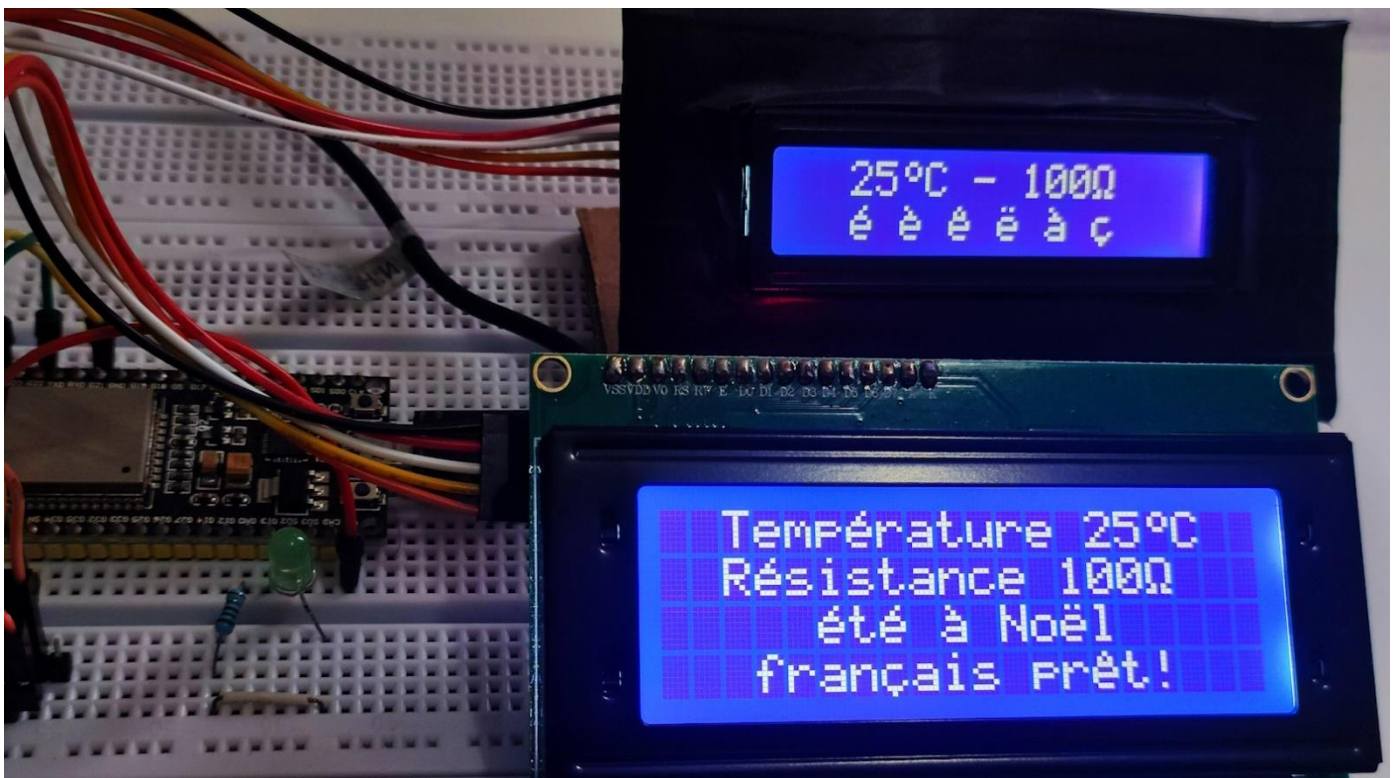
Program



Supported accented lowercase characters: é è ê ë à â ù û î ï ô ç. Degree (°) and omega (Ω) are also supported.

Supported uppercase accented characters (É È Ê Æ À Ù Î Ô Ç) use an ASCII fallback.

Result



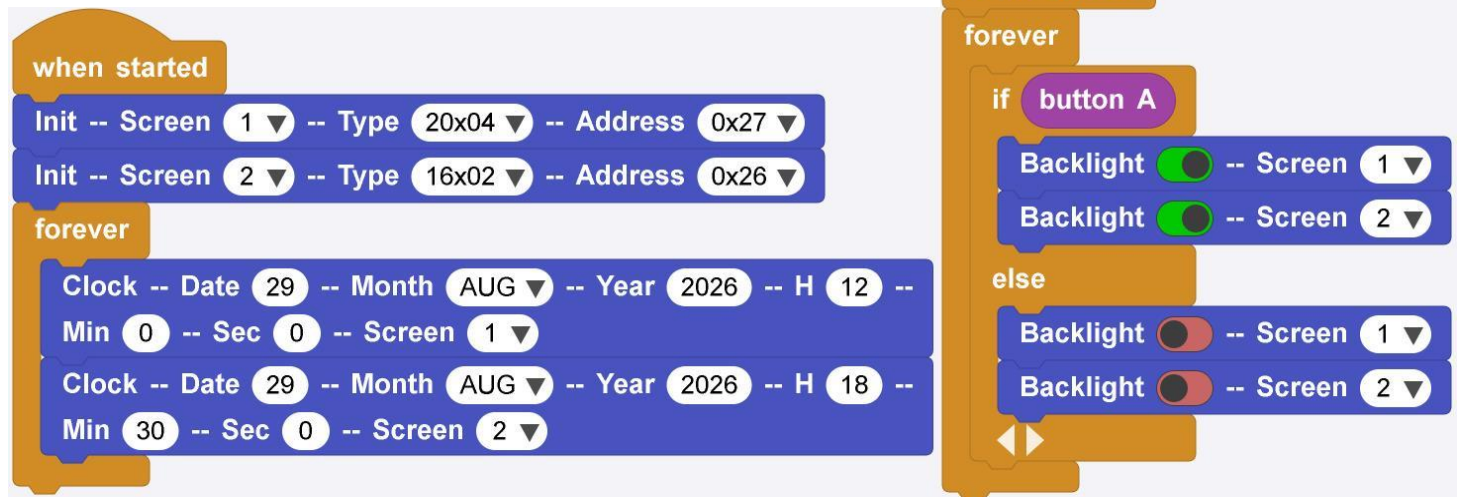
Example 6: Two Clocks, Backlight & Alarms

Example 6_a: Two Clocks & Backlight (06_a_Two_Clocks_Backlight.ubp).

Objective

Run two independent clocks and control the LCD backlights. Turning off the backlight when it is not required can reduce the power consumption of the display.

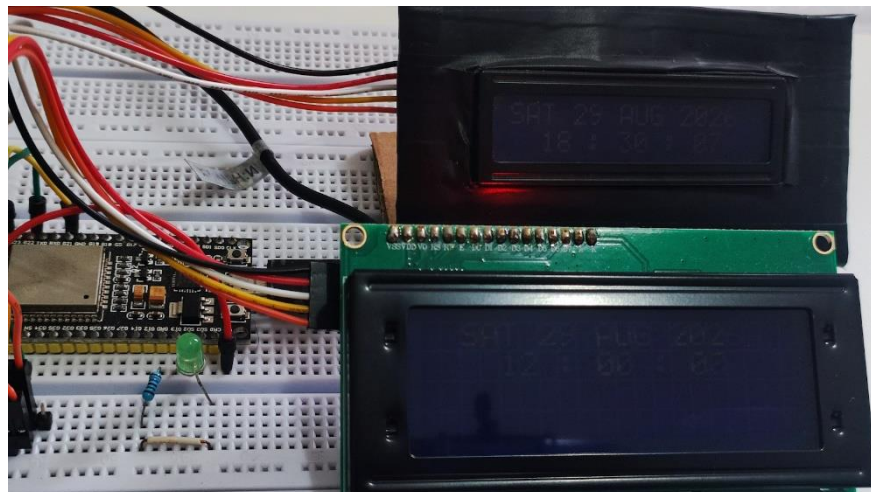
Program



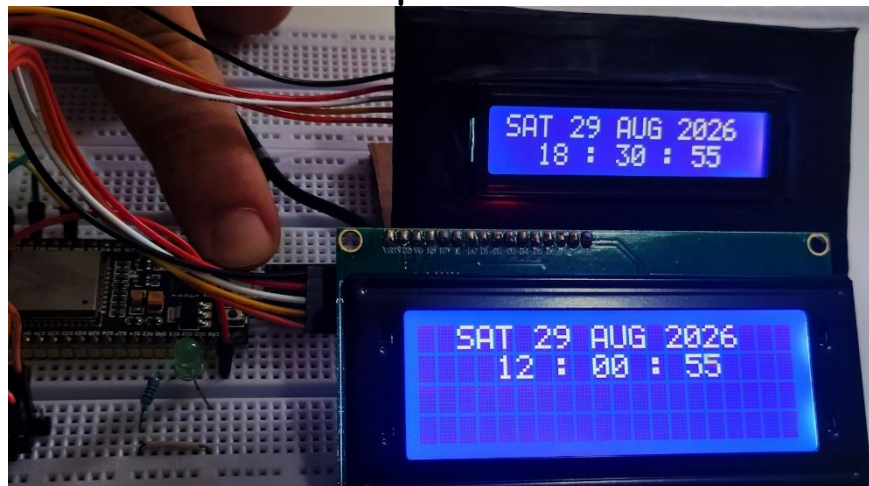
Note: The day of the week is calculated automatically from the date. Clock supports calendar validation and leap years from year 2000.

Result

Button released



Button pressed



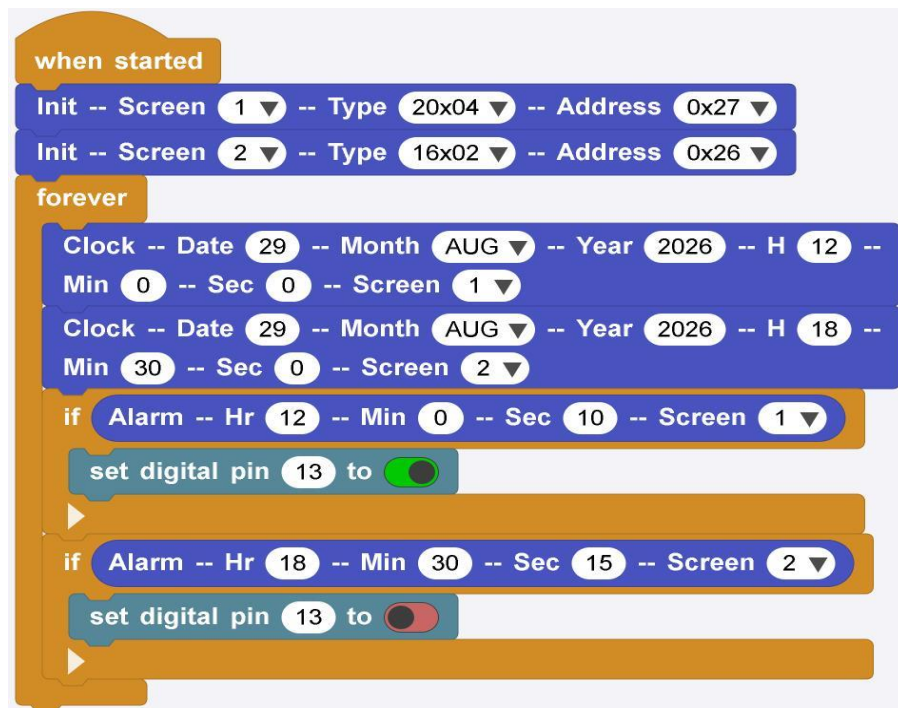
Example 6: Two Clocks, Backlight & Alarms

Example 6_b: Two Clocks & Alarms (06_b_Two_Clocks_Alarms.ubp).

Objective

Run two independent clocks and use an independent Alarm for each Screen.

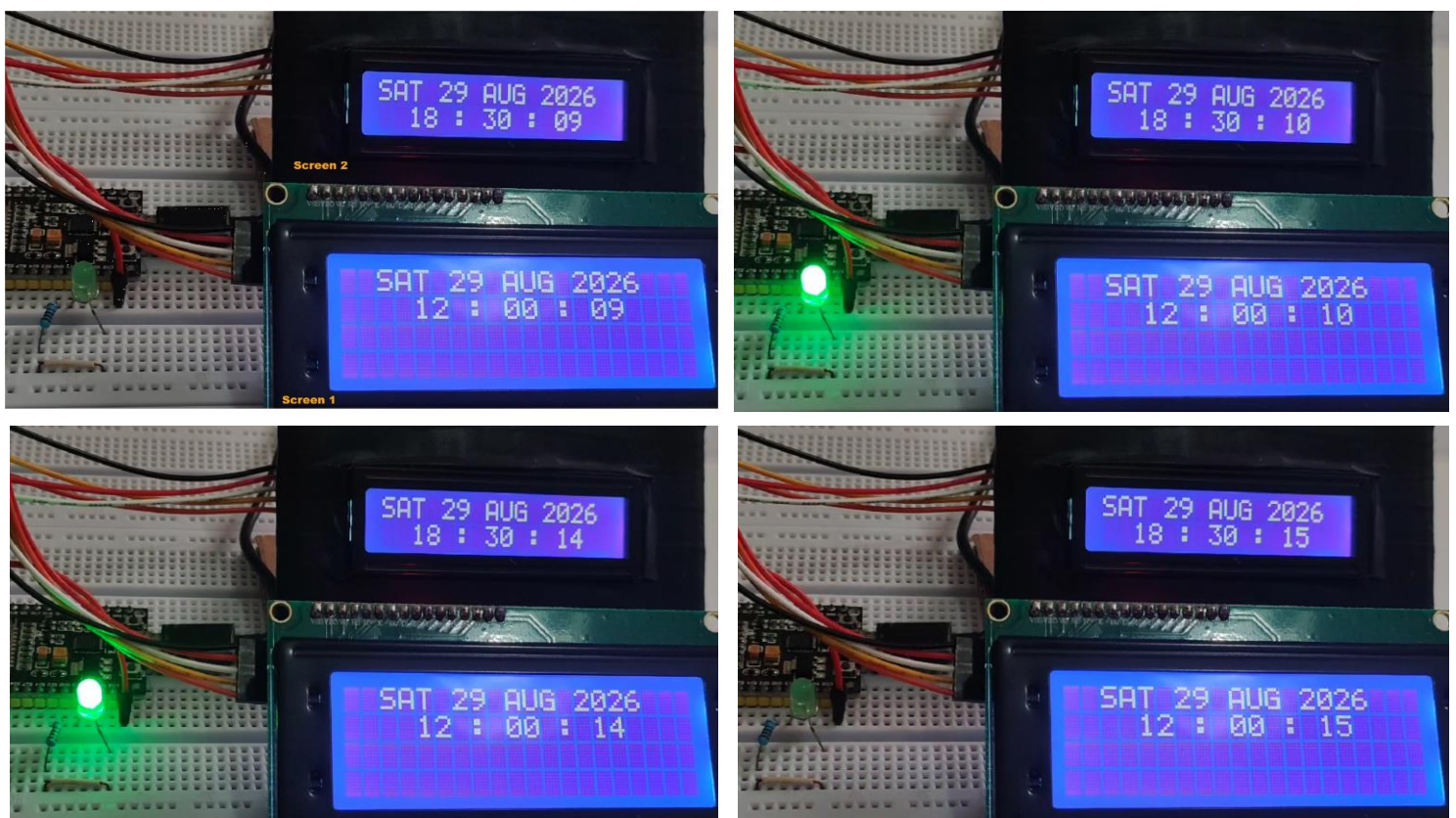
Program



Note: The two Alarms are independent. In this example, they control the same LED: the Screen 1 Alarm turns it ON, and the Screen 2 Alarm turns it OFF.

Try this: set Min to 60, or use 29 FEB 2026 in Clock. These values are rejected. 29 FEB 2028 is valid.

Result



Example 7: Countdown Control (07_Countdown_Control.ubp).

Objective

Use a physical button to control Countdown Start, Pause, Resume and restart with the same duration.

Program



Note: After FINISHED, press the button once to return to Ready with the same duration, then press it again to start a new cycle.

CtrlPin 0 is a valid GPIO pin; it does not mean 'disabled'. With Pull-up ON, the button is active LOW.

Result



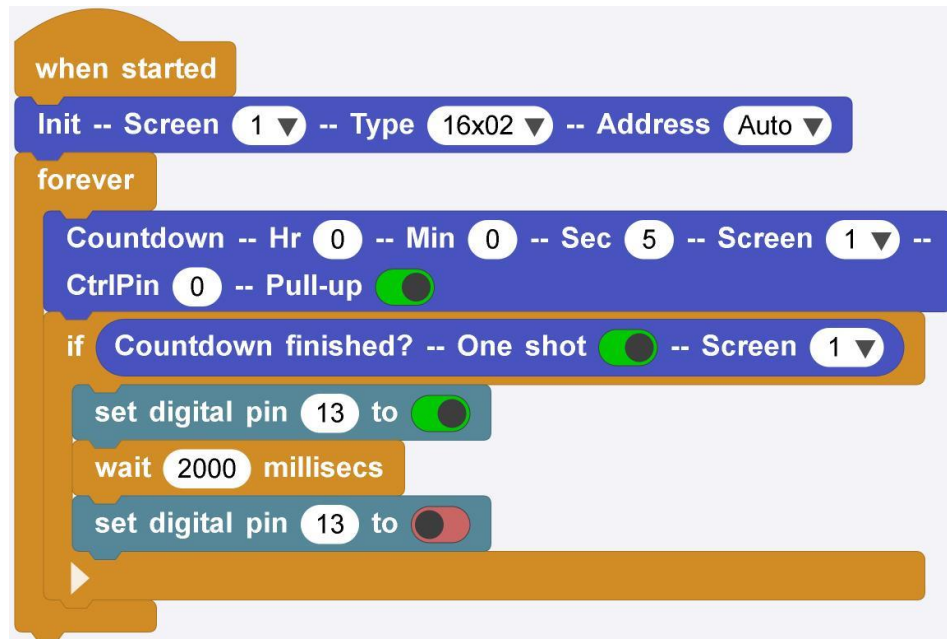
Example 8: Countdown One Shot

Example 8_a: One Shot ON (08_a_Countdown_One_Shot_ON.ubp).

Objective

Use Countdown finished? as a one-time event when the Countdown reaches zero.

Program



With One shot ON, Countdown finished? reports true only once for each completed countdown cycle. It is useful for triggering an action such as a buzzer, relay or timed LED.

Result

The LED turns ON when the countdown finishes.



The LED turns OFF after 2 seconds.



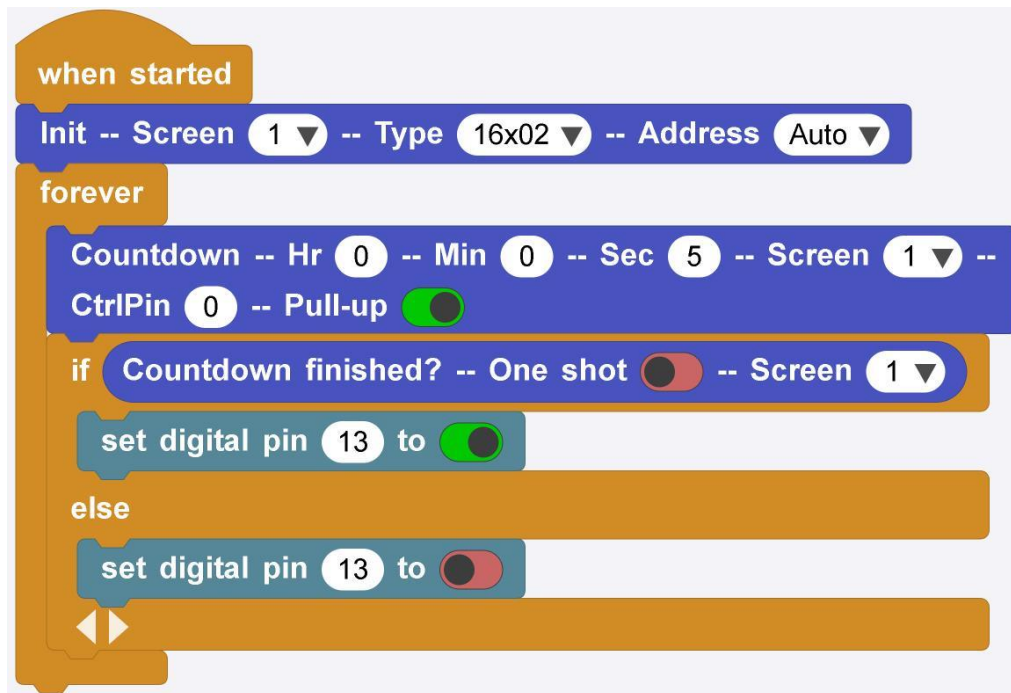
Example 8: Countdown One Shot

Example 8_b: One Shot OFF (08_b_Countdown_One_Shot_OFF.ubp).

Objective

Use Countdown finished? as a persistent FINISHED state.

Program



With One shot OFF, Countdown finished? remains true while the Countdown is in the FINISHED state. It is useful when you need to represent a persistent state.

Compare this example with Example 8_a to see the difference between an event and a state.

Result

The LED remains ON after the countdown finishes.

